Estructuras de Datos

Parcial – Librería – Parte 1

Primer semestre 2026

ANTES DE EMPEZAR

- Debe leerse primero el enunciado con cuidado (incorporando las partes de enunciado previas a cada ejercicio). Debe llevar entre 10 y 20 minutos.
- Cada ejercicio tiene estimado un mínimo y un máximo de tiempo que debería llevar suponiendo que se conoce el tema y fue bien practicado. Con esos tiempos, el examen debería llevar en total entre 1 hora y media (performance sin fallas) y 3 horas y media (yendo despacio, pero con seguridad). El tiempo total para el examen es de 4 horas.

Una librería (OJO: negocio de venta de libros, NO biblioteca, ni de funciones ni de libros) gestiona el catálogo de sus libros, cada uno identificado por un `Codigo` (un string, como, por ejemplo, el ISBN). De cada libro se registran el **stock disponible** (cantidad de unidades en depósito, un entero no negativo) y las **ventas acumuladas** desde que fue incorporado al catálogo. Las operaciones del sistema permiten registrar nuevos libros, reponer unidades, registrar ventas y consultar información al catálogo.

Para modelar el sistema se utilizará el TAD `Libreria`, cuya interfaz se da a continuación. Se denomina N a la cantidad de libros registrados y D a la cantidad de libros con stock disponible (D ≤ N).

```
type Codigo = String
type Cantidad = Int
```

- `nuevaLibreria :: Libreria`, que describe una librería sin libros.
  Eficiencia: *O(1)*
- `registrar :: Codigo -> Libreria -> Libreria`, que describe la librería que resulta de incorporar el libro de código dado, con stock 0 y ventas 0 a la librería dada.
  Precondición: el libro no está registrado.
  Eficiencia: *O(log N)*
- `reponer :: Codigo -> Cantidad -> Libreria -> Libreria`, que describe la librería que resulta de sumarle la cantidad dada al stock del libro dado en la librería dada.
  Precondición: el libro está registrado, la cantidad dada es mayor que cero.
  Eficiencia: *O(log N)*
- `vender :: Codigo -> Libreria -> Libreria`, que describe la librería que resulta de descontar una unidad del stock del libro dado e incrementar en una unidad las ventas acumuladas de ese libro en la librería dada.
  Precondición: el libro está registrado y tiene stock mayor que cero.
  Eficiencia: *O(log N)*
- `stockDe :: Codigo -> Libreria -> Int`, que describe el stock disponible del libro en la librería dada.
  Precondición: el libro está registrado.
  Eficiencia: *O(log N)*

- `disponibles :: Libreria -> [Codigo]`, que describe la lista de libros con stock mayor que cero en la librería dada.
  Eficiencia: *O(D)*
- `masVendido :: Libreria -> Codigo`, que describe el código del libro con más ventas acumuladas en la librería dada; en caso de empate, cualquiera de los que tengan más ventas.
  Precondición: al menos un libro tiene ventas mayores a cero.
  Eficiencia: *O(N log N)*

RECORDATORIO: No olvidar agregar las precondiciones que sean necesarias, dividir en subtareas cuando ayude a la claridad, e incluir propósito y precondiciones de las funciones auxiliares que se programen. También debe recordarse que es importante establecer la eficiencia y justificarla. También tener en cuenta adecuadamente la parcialidad de las funciones de interfaz del TAD.

Ejercicio 1)

Implementar como usuario del TAD `Libreria` la función `conMasStockQueElMasVendido :: [Codigo] -> Libreria -> [Codigo]`, que, dada una lista de códigos y una librería, devuelve los libros de la lista cuyo stock supera estrictamente al stock del libro más vendido. **Precondición:** todos los códigos de la lista están registrados en la librería y al menos un libro tiene ventas mayores a cero.

> **ENUNCIADO**
>
> Dado el siguiente tipo de representación para implementar el TAD `Libreria` en Haskell:
>
> ```
> data Libreria = Lib (Map Codigo (Int, Int))  -- (stock, ventas) por libro
>                     (Set Codigo)             -- libros con stock > 0
> ```
>
> Donde el map asocia a cada libro su par (stock disponible, ventas acumuladas). Y el set contiene exactamente los códigos de los libros cuyo stock es estrictamente mayor a cero.
>
> **Observación:** la información del set puede derivarse del map; la redundancia persigue fines didácticos.

Ejercicio 2)

Escribir los invariantes de representación para poder crear elementos válidos del TAD.

Ejercicio 3)

Implementar las funciones `vender` y `masVendido` de la interfaz.

Anexo de interfaces

Map, siendo k la cantidad de claves distintas en el map:	
`emptyM :: Map k v`	O(1)
`assocM :: Ord k => k -> v -> Map k v -> Map k v`	O(log k)
`lookupM :: Ord k => k -> Map k v -> Maybe v`	O(log k)
`deleteM :: Ord k => k -> Map k v -> Map k v`	O(log k)
`keysM :: Eq k => Map k v -> [k]`	O(k)
`sizeM :: Map k v -> Int`	O(1)

Set, siendo n la cantidad de elementos del conjunto:	
`emptyS :: Set a`	O(1)
`isEmptyS :: Set a -> Bool`	O(1)
`addS :: Eq a => a -> Set a -> Set a`	O(log n)
`deleteS :: Eq a => a -> Set a -> Set a`	O(log n)
`belongsS :: Eq a => a -> Set a -> Bool`	O(log n)
`sizeS :: Set a -> Int`	O(1)
`set2list :: Set a -> [a]`	O(n)

(SIGUE EN LA PARTE 2)

Estructuras de Datos

Parcial – Librería – Parte 2

Primer semestre 2026

(VIENE DE LA PARTE 1)

La misma interfaz del TAD `Libreria` se implementa en C++ con los cambios necesarios en la interfaz para que las operaciones sean imperativas. El propósito y precondiciones son los mismos, pero cambia el hecho de que ahora es modificación in-place de la estructura, y las eficiencias son diferentes.

ACLARACIÓN: por efectos prácticos y porque debe resolverse en papel, los parámetos se abrevian con una sola letra. Sin embargo, esto NO es una práctica recomendable en general.

```
// Libreria.h
typedef string Codigo;
typedef int Cantidad;
struct LibreriaStr;
typedef LibreriaStr* Libreria;

Libreria nuevaLibreria();                          // O(1)
void registrar(Codigo c, Libreria l);              // O(1)
void reponer(Codigo c, Cantidad n, Libreria l);    // O(N)
void vender(Codigo c, Libreria l);                 // O(N)
int stockDe(Codigo c, Libreria l);                 // O(N)
Codigo masVendido(Libreria l);                     // O(N)
```

Ejercicio 4)

Implementar como usuario del TAD Librería la función

```
bool todosConMasStockQueElMasVendido(Codigo cs[], int n, Libreria l)
```

que dado un array cs de n códigos y una librería, indica si todos los libros del array tienen stock estrictamente mayor al del libro más vendido. **Precondición:** todos los códigos están registrados en la librería, y al menos un libro tiene ventas mayor a cero. Comparar la eficiencia de esta función con la eficiencia obtenida en el **Ejercicio 1** y explicar la diferencia.

ENUNCIADO

Dado el siguiente tipo de representación en `Libreria.cpp` que utiliza arrays dinámicos paralelos:

```
#define CAPACIDAD 8

struct ChunkStr {
    string cod[CAPACIDAD]; // códigos
    int stk[CAPACIDAD];    // stocks (paralelo a cod)
    int ven[CAPACIDAD];    // ventas (paralelo a cod)
    int cant;              // libros en este chunk
    ChunkStr* next;        // siguiente chunk
};
typedef ChunkStr* Chunk;

struct LibreriaStr {
    Chunk pri;  // primer chunk
    Chunk ult;  // último chunk
    int cant;   // total de libros registrados
};
```

La representación es una lista simplemente encadenada de **chunks**, donde cada chunk contiene hasta CAPACIDAD códigos de libros en arrays estáticos paralelos. Los libros se acumulan de izquierda a derecha dentro de cada chunk; cuando el último se llena, se encadena uno nuevo, sin necesidad de relocalizar nada. Por ejemplo, si la CAPACIDAD es 8, una librería con 7 libros tiene un único **chunk** Ch que tiene los códigos en sus primeras 7 posiciones. Si se agrega un nuevo libro, se registra en el mismo **chunk** en la última posición. Si se agrega otro más, el sistema reserva un nuevo **chunk** Ch', lo encadena luego de Ch, y registra el código en la primera posición de Ch'.

Ejercicio 5)

Definir los invariantes de representación para `LibreriaStr` y `ChunkStr`.

Ejercicio 6)

Implementar las funciones `registrar` y `masVendido` de la interfaz en C++, respetando las restricciones de eficiencia pedidas y justificando los costos.

Ejercicio 7)

Responder en palabras las siguientes preguntas justificando debidamente.

1. ¿Cómo debería modificarse la implementación para conseguir que las operación `masVendido` sea O(1)? ¿Cómo afectaría este cambio a las funciones de los ejercicios 1 y 4?
2. ¿Cuál es la diferencia entre el invariante de representación de un AVL y un árbol binario que representa un Heap? Recordar que ambos árboles están balanceados. ¿Qué es lo que permite hacer rápidamente un AVL que un Heap no, y por qué? ¿Cuál es la operación que es más eficiente en un Heap que en un AVL, y por qué?

Anexo de interfaces

Map, siendo K la cantidad de claves distintas en el map, y K y V tipos específicos declarados:	
`Map emptyM()`	O(1)
`V assocM(K k, V v, Map m)`	O(log K)
`bool memberM(K k, Map m)`	O(log K)
`V lookupM(K k, Map m) // PREC: memberM(k,m)`	O(log K)
`void deleteM(K k, Map m)`	O(log K)
`V[] keysM(Map m)`	O(K)

Set, siendo N la cantidad de elementos del conjunto, y A un tipo especifico declarado.:	
`Set emptyS()`	O(1)
`bool isEmptyS(Set s)`	O(1)
`void addS(A x, Set s)`	O(log N)
`void deleteS(A x, Set s)`	O(log N)
`bool belongsS(A x, Set s)`	O(log N)
`int sizeS(Set s)`	O(1)

Nota: en las imágenes llegaban hasta la página 6 de 7. La página 7 no estaba incluida.